

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 2 — Progettare il prompt

System, user, developer

Chi scrive che cosa

Un prompt, in una conversazione: tre voci, tre ruoli

In una conversazione con Claude il testo ha tre autori: **chi costruisce l'applicazione** scrive il **system prompt**, **chi la usa** il **messaggio**, **Claude** la risposta — e il prompt è tutto ciò che il modello riceve prima di rispondere



Nella **Claude API** il system prompt è un parametro a parte, `system`: contesto e istruzioni, come un obiettivo o un ruolo; la conversazione è fatta di turni alternati, `user` e `assistant`

Sui modelli di oggi — Haiku, Sonnet e Opus 5.5 compresi — chi costruisce l'applicazione può aggiungere messaggi `system` anche **a metà conversazione**, dopo un turno `user`: per Claude vengono da chi costruisce l'applicazione, non dall'utente, e in un conflitto **prevalgono**

E il developer message?

La stessa voce, nomi diversi da un fornitore all’altro

- Nella documentazione di **OpenAI** i messaggi `developer` portano le istruzioni di chi sviluppa l’applicazione e hanno la **precedenza** sui messaggi `user`; dai modelli o1 hanno preso il posto dei messaggi `system`
- Nella documentazione di **Google**, per la Gemini API, la stessa voce si chiama *system instructions*
- **Su Claude** «developer» compare nel livello di compatibilità con l’SDK di OpenAI: lì i messaggi `system` e `developer` finiscono nel **system prompt**, ovunque siano nella conversazione

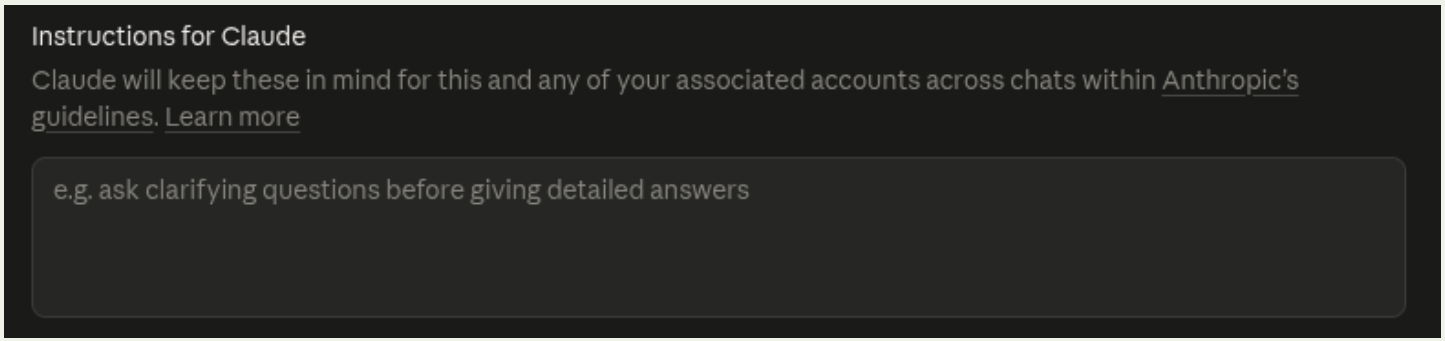
Chi scrive	OpenAI	Claude API	Claude Code	claude.ai
Chi costruisce l’applicazione	messaggi `developer`	`system`, anche a metà conversazione	il system prompt di Claude Code, o il vostro con un’opzione	il system prompt di Anthropic
Chi la usa	messaggi `user`	messaggi `user`	il messaggio, nel terminale	il messaggio, e le sue *Instructions for Claude*

«Developer» è il nome che OpenAI dà ai messaggi di chi sviluppa l’applicazione: per Claude sono il system prompt.

Il system prompt che non vedete

In claude.ai e in Claude Code c’è sempre: M2-D02 lo pesa

- **claude.ai**: un system prompt di Anthropic, che dà a Claude informazioni aggiornate come la data; Anthropic lo **pubblica** e lo aggiorna periodicamente. E ci sono i livelli che aggiungete voi: le *Instructions for Claude* (tutte le conversazioni), le istruzioni di un progetto (solo le sue chat), le skill (come Claude formatta e consegna le risposte)



Le impostazioni di claude.ai: le *Instructions for Claude*, istruzioni che valgono per tutte le conversazioni
Schermata di claude.ai del 20 ottobre 2026, dall’account del docente

Il system prompt che non vedete

In claude.ai e in Claude Code c’è sempre: M2-D02 lo pesa

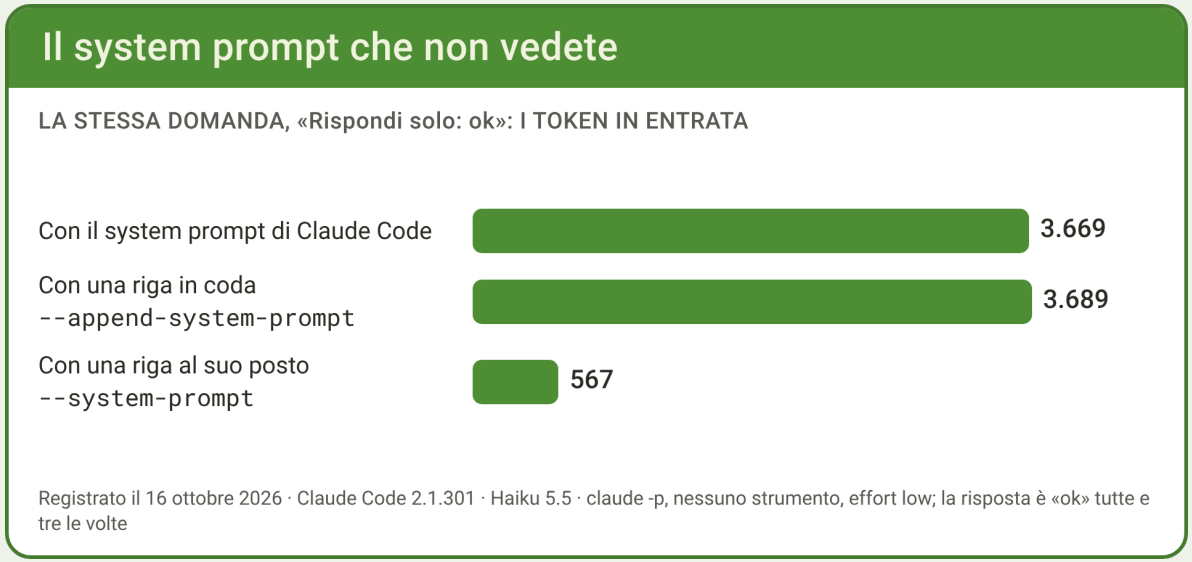
- **Claude Code**: il suo system prompt — uso degli strumenti, comportamento sicuro, formato delle risposte — parte a ogni richiesta, anche con `claude -p`, anche se non scrivete nulla. Quanto pesa? Demo **M2-D02**: la stessa domanda di una riga, tre volte su Haiku

```
# nello spazio di lavoro di tools/prepara_esercizio.sh M2-D02
chiedi() {
  claude -p "$(cat domanda.txt)" --model haiku --effort low \
    --tools "" --strict-mcp-config --output-format json "$@"
}
p="$(chiedi)"
a="$(chiedi --append-system-prompt "$(cat istruzione.txt)")"
s="$(chiedi --system-prompt "$(cat istruzione.txt)")"
```

- Guardate i **token in entrata**: con il system prompt di Claude Code; con `istruzione.txt`, «Rispondi in italiano.», **in coda**; con la stessa riga **al suo posto**. La domanda, `domanda.txt`: «Rispondi solo: ok». Il passo 3 del README mette i numeri in `sistema.tsv`

Il system prompt che non vedete

In claude.ai e in Claude Code c’è sempre: M2-D02 lo pesa



Haiku, effort low, tre chiamate con la stessa domanda: i token in entrata
Tre volte «ok»: cambia solo ciò che arriva prima della domanda
© Simone Scannapieco — figura dalla registrazione del 16 ottobre 2026

- Stessa risposta: cambia ciò che il modello riceve **prima** della domanda — oltre **tremila token** di istruzioni che non avete scritto. Non è un testo fisso: per i modelli più avanzati Anthropic ne ha tolto **oltre l’80 %**, senza perdite misurabili nelle sue valutazioni di coding
- `--append-system-prompt` **aggiunge** in coda; `--system-prompt` **sostituisce** tutto, comprese le indicazioni sugli strumenti e le istruzioni di sicurezza: ciò che serve, lo rimettete voi
- `CLAUDE.md` **non** è nel system prompt: arriva come messaggio, subito dopo

Che cosa va dove

Il system prompt per ciò che resta, il messaggio per il caso

A voce: il **tono di voce** dell'azienda — nel system prompt o nel messaggio? E il **testo del cliente**?

Ingrediente	Dove	Perché
Il ruolo e le regole stabili: tono, lunghezza, che cosa non promettere	**system prompt**	non cambia da una richiesta all'altra: è configurazione
Il caso: l'istruzione, il formato, i vincoli del caso	**messaggio**	cambia a ogni richiesta
I dati: il testo del cliente, il log da leggere	**messaggio**, tra tag	sono la variabile, non il programma
Gli esempi	**messaggio**, ognuno nel suo tag	mostrano la forma, non sono regole

- L'esempio di Anthropic: il ruolo nel `system`, una frase sola; la domanda nel messaggio `user`
- In **Claude Code**: `--append-system-prompt` in uno script che resta Claude Code con regole in più; `--system-prompt`, o il file di un agente, per un altro strumento, come uno che analizza i log
- In **claude.ai**: le *Instructions for Claude*, o le istruzioni del progetto

Ciò che non cambia da una richiesta all'altra va nel system prompt; il caso e i dati, nel messaggio.